SQL «part 4»

(More Advanced Stuff)

- Transactional Systems
- Interaction with Programming Languages
- Generic Integrity Constraints
- Stored Procedures
- Active Rules

Transactional Systems

(Concurrency, ACID properties)

Transactional Systems

Definition of Transaction

- An elementary, atomic unit of work performed by an application
- Each transaction is encapsulated within two commands:
  - **`begin transaction`** (bot)
  - **`end transaction`** (eot)
- Within a transaction, **one** of the commands below is executed (exactly **once**):
  - **`commit work`** (commit)
  - **`rollback work`** (abort)
- **On-Line transactional System** (OLTP): a system that supports the execution of transactions on behalf of concurrent applications

Transactions: an example

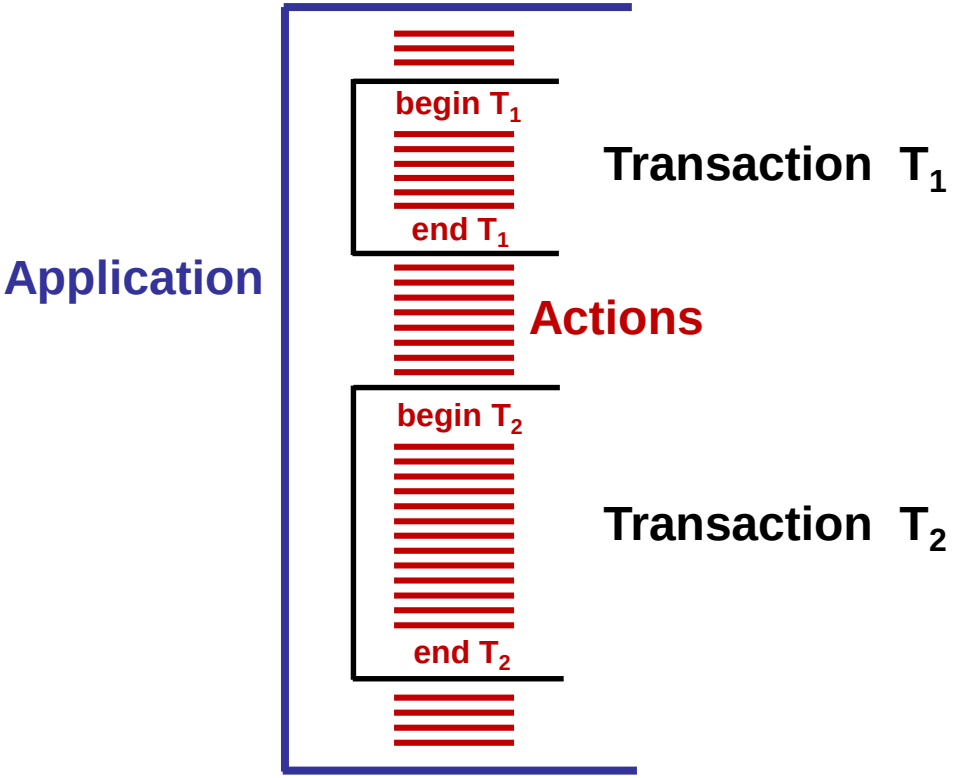
```
begin transaction;
update Account
   set Balance = Balance + 10 where AccNum = 12202;
update Account
   set Balance = Balance – 10 where AccNum = 42177;
commit-work;
end transaction;
```

Transactions: an example with alternatives

```
begin transaction;
update Account
   set Balance = Balance + 10 where AccNum = 12202;
update Account
   set Balance = Balance – 10 where AccNum = 42177;
select Balance into A from Account
   where AccNum = 42177;
if ( A >= 0 ) then commit-work
                 else rollback-work;
end transaction;
```

Transactional Systems

Difference between Application and Transaction



Transactional Systems

Well-formed Transactions

begin transaction

code for data manipulation (reads and writes on DB)

commit-work / rollback-work

possibly more code, but no data manipulation

end transaction



Transactional Systems

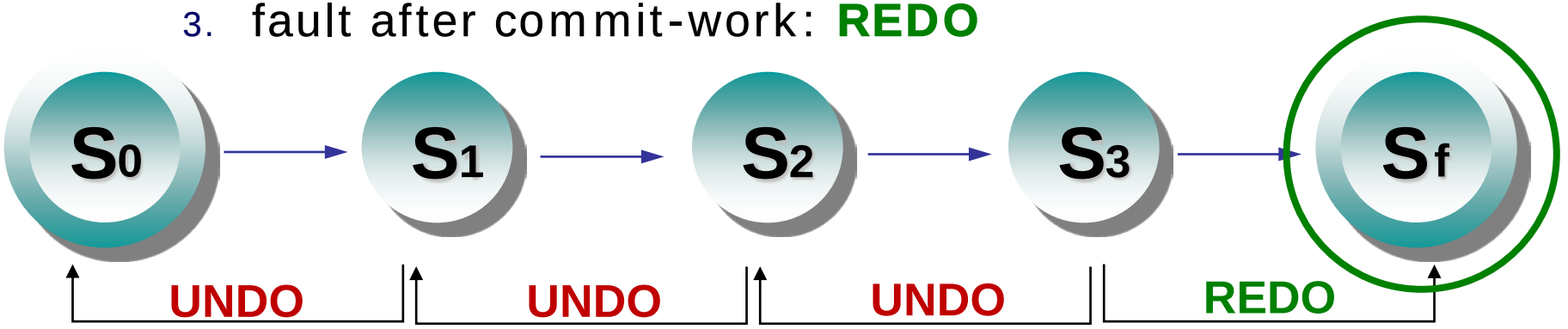
ACID Properties of Transactions

- A transaction is a unit of work enjoying the following properties:
  - Atomicity
  - Consistency
  - Isolation
  - Durability

Transactional Systems

Atomicity

- A transaction is an **atomic** transformation from an initial state to a final state
- Possible behaviors:
  1. commit-work: **SUCCESS**
  2. rollback-work or fault before commit-work: **UNDO**
  3. fault after commit-work: **REDO**



Transactional Systems

Consistency

- A transaction must satisfy the DB integrity constraints

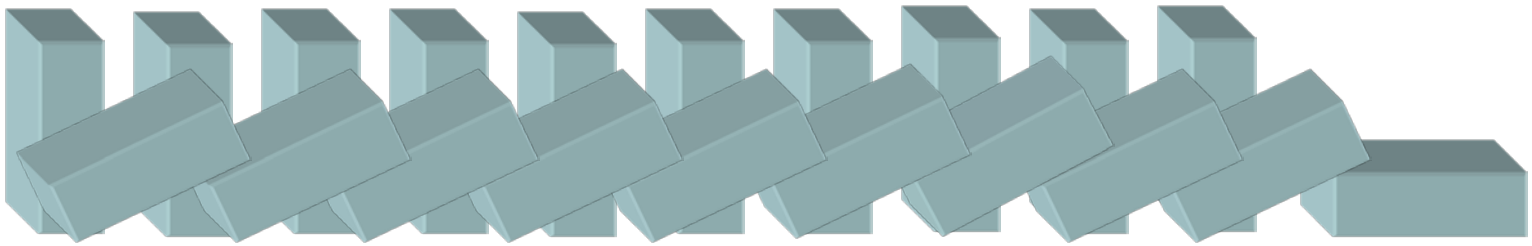
- Consequence:
  - **if** the initial state S_0 is consistent
  - **then** the final state S_f is also consistent

This is not necessarily true for the intermediate states S_i

Transactional Systems

Isolation

- A transaction is not affected by the behavior of other concurrent transactions
- Consequence:
  - Its intermediate states are not exposed
  - The "domino effect" is avoided (there is no risk that the rollback of a transaction causes that of others)



Transactional Systems

Durability

- The effect of a transaction that has successfully committed will last "**forever**"
  - Independently of any system fault

This is less obvious than it might sound... data manipulation always occurs in MAIN MEMORY, and propagation to mass memory is typically delayed

Transactional Systems

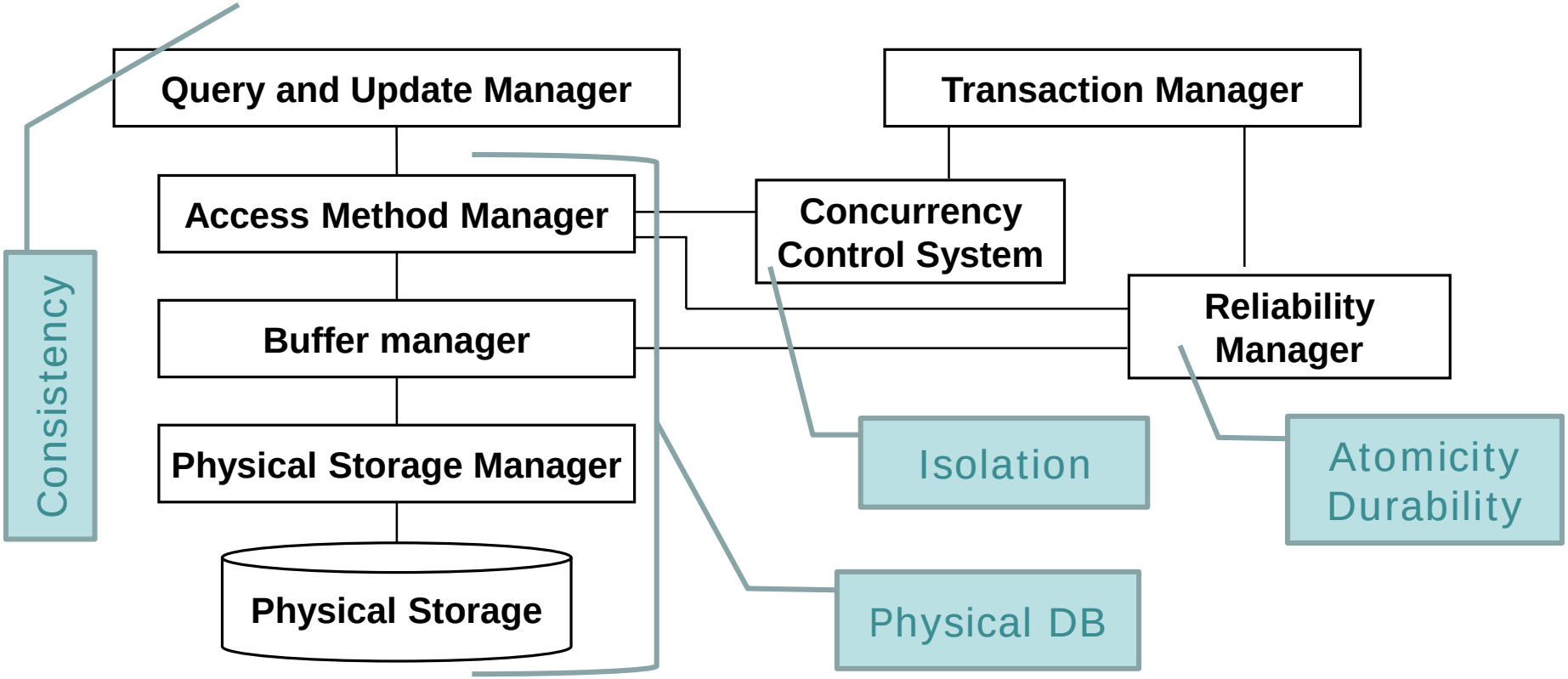
Transaction Properties and related mechanisms

- **A**tomicity
  - Abort-rollback-restart, Commit protocols
- **C**onsistency
  - Integrity checking of the DBMS
- **I**solation
  - Concurrency control
- **D**urability
  - Recovery management

ACID properties

Transactional Systems

Logical Architecture of a DBMS



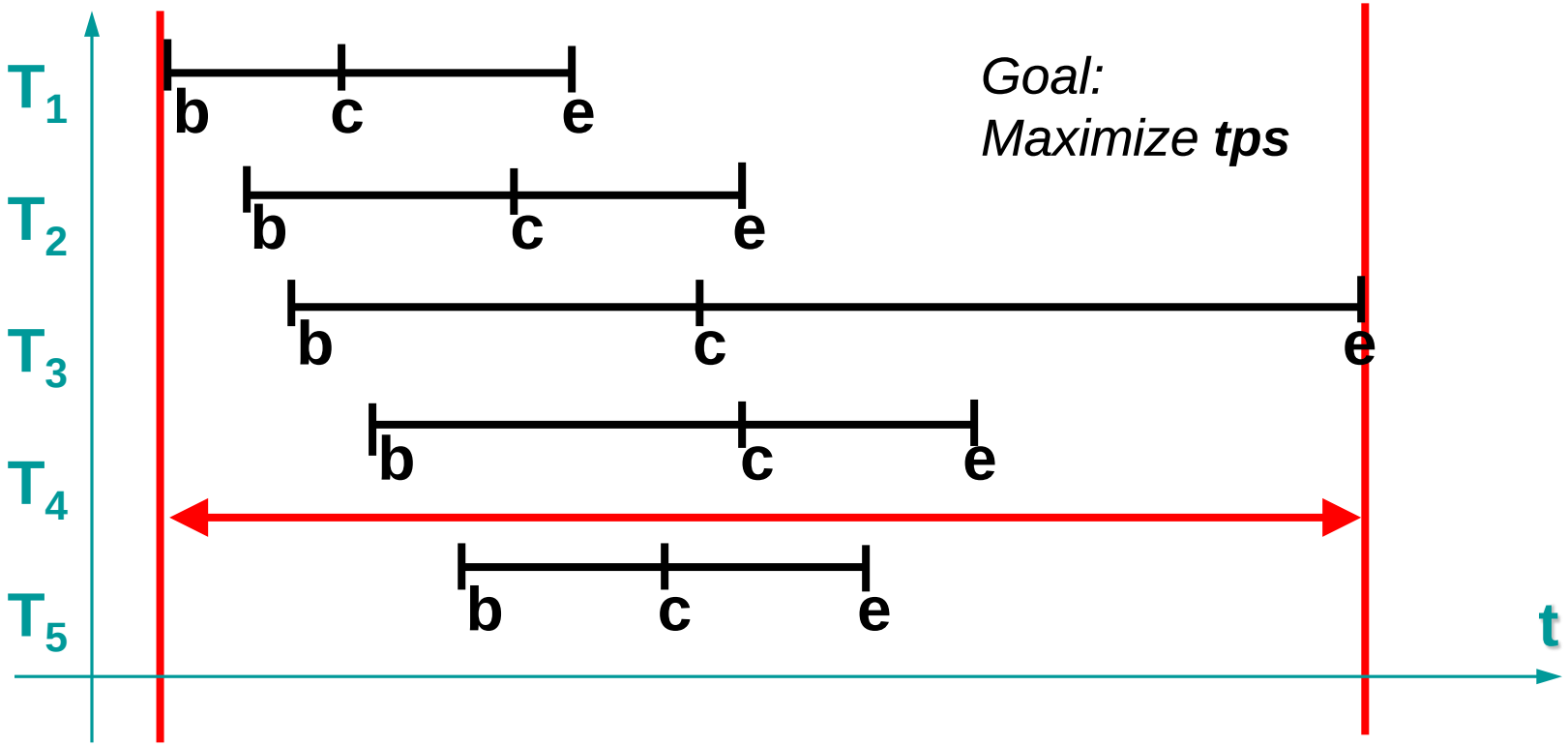
Transactional Systems

Transactions and DBMS modules

- **Atomicity** and **Durability**
  - Reliability Manager
- **Isolation**
  - Concurrency Control System
- **Consistency**
  - Integrity Control System at query execution time (with the support of the DDL compiler)

Transactional Systems

Advantages of Concurrency



Transactional Systems

Concurrency Control

- Concurrency is fundamental
  - Tens or hundreds of transactions per second cannot be executed serially
- Examples: banks, ticket reservations

- **Problem**: concurrent execution may cause **anomalies**
  - Concurrency needs to be controlled

Transactional Systems

Problems due to Concurrency

T_1:
```
begin transaction
update account
 set balance = balance + 3
 where customer = 'Smith'
commit work
end transaction
```

T_2:
```
begin transaction
update account
 set balance = balance + 6
 where customer = 'Smith'
commit work
end transaction
```

Concurrent SQL transactional statements addressing the same resource

T_1 :
```
begin transaction
D = D + 3
commit work
end transaction
```

T_2 :
```
begin transaction
D = D + 6
commit work
end transaction
```

Lower level view of the transactions

T_1 :
```
begin transaction
read(D,x)
x = x + 3
write(x,D)
commit work
end transaction
```

T_2 :
```
begin transaction
read(D,y)
y = y + 6
write(y,D)
commit work
end transaction
```

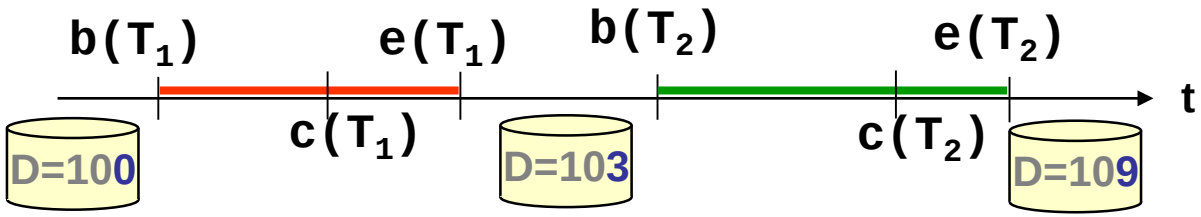
Transactional Systems

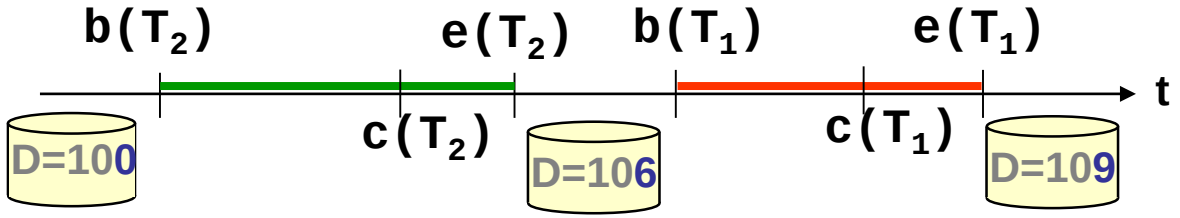
Serial Executions

```
bot(T1)
 r(D,x)
 x = x + 3
 w(x,D)
commit(T1)
eot(T1)
```

```
bot(T2)
 r(D,y)
 y = y + 6
 w(y,D)
commit(T2)
eot(T2)
```

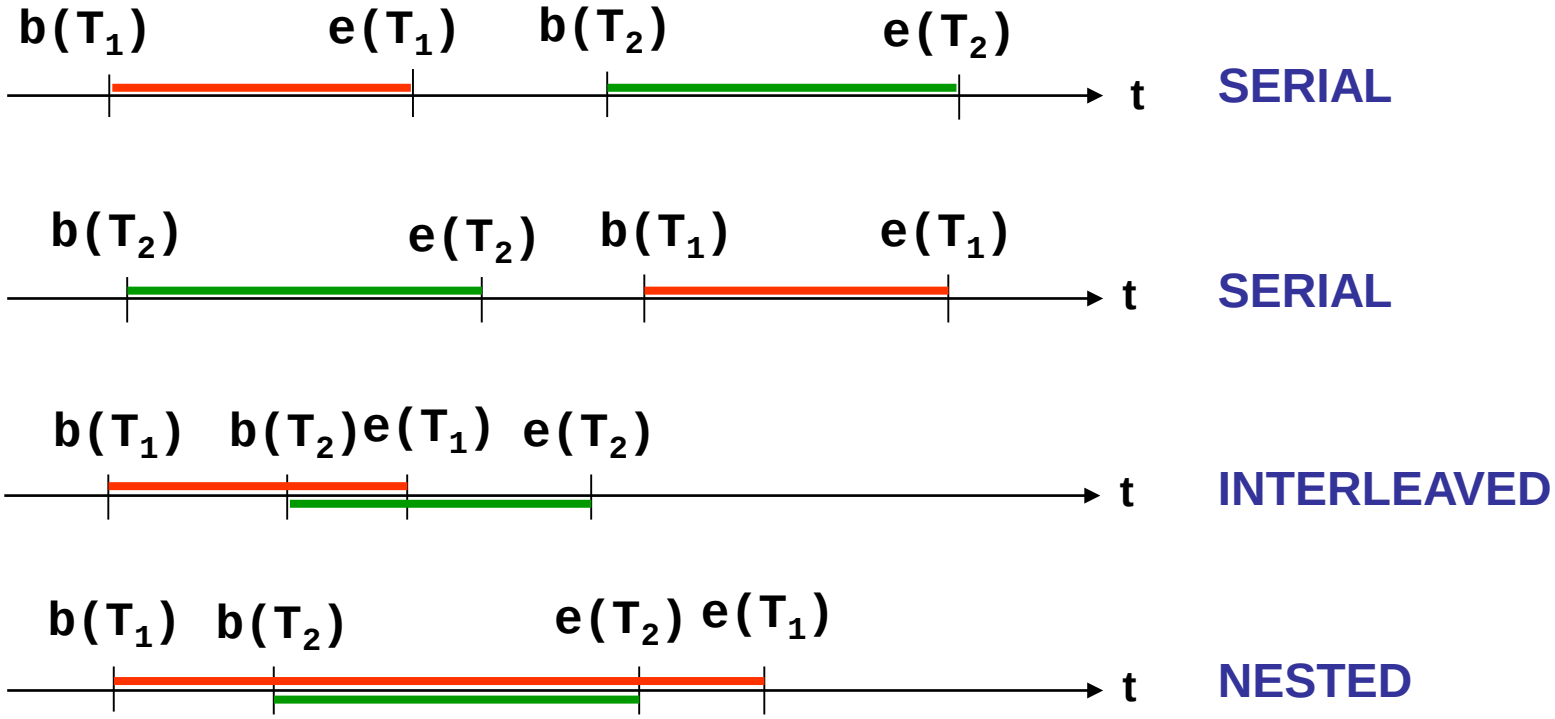
Initially D_0=100





Transactional Systems

Concurrent Executions



Execution with "Lost Update"

D = 100

1	T_1: r(D,x)	
2	T_1: x = x + 3	
3	T_2: r(D,y)	
4	T_2: y = y + 6	
5	T_1: w(x,D)	D = 103
6	T_2: w(y,D)	D = 106 !
7	T_1: commit	
8	T_2: commit	

Sequence of operations producing the anomaly



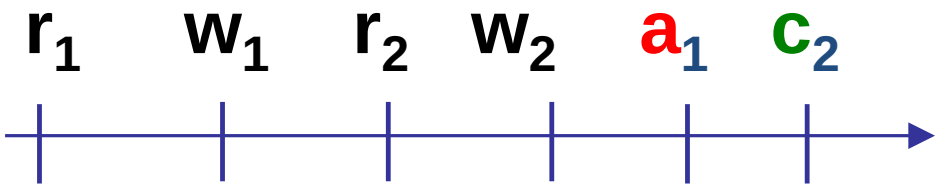
Execution with "Dirty Read"

D = 100

T_1 now aborts instead of committing

1	T_1: r(D,x)	
2	T_1: x = x + 3	
3	T_1: w(x,D)	D = 103
4	T_2: r(D,y)	
5	T_1: **abort**	
6	T_2: y = y + 6	
7	T_2: w(y,D)	D = 109 !
8	T_2: commit	

Sequence of operations producing the anomaly



Transactions and DBMS modules

- **Atomicity** and **Durability**
  - Reliability Manager
- **Isolation**
  - Concurrency Control System
- **Consistency**
  - Integrity Control System at query execution time (with the support of the DDL compiler)

JDBC: creation of a connection to the DB

```
Class.forName("sun.jdbc.odbc.JdbcOdbcDriver");
url = "jdbc:odbc:MyConnection"
Connection con = DriverManager.getConnection( url, "myLogin",
                                                "myPassword" );
 ...  // Use of the connection to execute statements

con.close();
```

JDBC: execution of a query statement

```
String query = "select * from COFFEES";

Statement stmt = con.createStatement();
ResultSet rs = stmt.executeQuery( query );

while( rs.next() ) {
 String s = rs.getString("COFFEE_NAME");
 float n = rs.getFloat("PRICE");
 System.out.println( s + " " + n );
}

rs.close();
stmt.close();
```

ResultSets always accessed via **rs.next()**

```
query = "select count(*) AS C from T";
rs = stmt.executeQuery(query);

if( rs.next() )
  int m = rs.getString("C");
else
  System.out.println("Ahhhhhhh");
```

JDBC: execution of an update statement

```
Statement stmt = con.createStatement();
stmt.executeUpdate( "insert into COFFEES " +
                    "values( 'Colombian', 101, 7.99, 0, 0 )" );

... // Possible reuse of stmt for other commands

stmt.close();
```

JDBC: data manipulation is prone to failures

Data manipulation commands on a DB are prone to failures

(*constraint violations, authentication failures, timeouts, ...*)

and shall always be encapsulated in try/catch blocks

```
try
 {  ... /* Execution of a statement */    }
catch(SQLException se) {
   ... // Do something (clever)
 }
catch(Exception e) {
   ... // Do something else (still clever)
 }
```

Transactions in JDBC

- The ***transaction mode*** is set with a method defined in the **Connection** interface

  setAutoCommit(boolean autoCommit)

- **con.setAutoCommit(true)**
  - (***default***) "autocommit": every single operation is a transaction
- **con.setAutoCommit(false)**
  - Transactions are explicitly handled by the program
    - **con.commit()**
    - **con.rollback()**
  - There is no **begin_transaction** command

JDBC: an example of transaction

```
try {
        con.setAutoCommit( false );
        Statement stmt = con.createStatement();
        stmt.executeUpdate( "…" );
        stmt.executeUpdate( "…" );
        …
        con.commit();
}
catch( Exception ex ) {
        try { con.rollback();}
        catch( SQLException sqex ) { ... }
}
```

Consistency and Generic Constraints

- **Atomicity** and **Durability**
  - Reliability Manager
- **Isolation**
  - Concurrency Control System
- **Consistency**
  - Integrity Control System at query execution time (with the support of the DDL compiler)

Generic Integrity Constraints

- Predicates that must always be «true» (satisfied) by any instance of the schema

- Expressed in two ways, according to the standard:
  - within the definition of tables (widely supported)
  - with separate assertions (rather neglected by implementations)

- **Basic syntax**: **check** (*Condition*)

  Condition is an arbitrarily complex expression that evaluates to true/false

Example: Inventory Management

```
create table Warehouse as
  ( ProdId      char(2) primary key,
    AvQty       integer not null
                 check(AgvQty > 0),
    ReordLimit integer not null
                 check(ReordLimit > 10),
    check(AvQty > ReordLimit)
  )
```

check could be a silver bullet...

```
create table Employee (

   EmpId char(6) check ( EmpId is not null and
                         1 = ( select count(*)
                               from Employee E
                               where EmpId = E.EmpId ) ),

   LastName varchar(20) check (LastName is not null),

   Name varchar(20) check ( Name is not null and
                            2 > ( select count(*)
                                  from Employee E
                                  where (Name,LastName) = (E.Name,E.LastName)),

   Depart character(15) check ( Depart in ( select DepId
                                            from Department ))
 )
```

```
create table Exam
 (  sid       char(6),
    cid       char(6),
    Date      date  not null,
    grade     smallint not null,
    primary key(sid,cid),
    check (grade between 18 and 30) )
```

Very specific constraints

The manager of all employees must be in their same department, with the exceptions of employees whose EmpId starts with 1

Extending the table definition with this "**pseudo-referential integrity**":

```
create table Employee (
…
MgrId char (6)
    check ( EmpId like "1%" or
            Depart = ( select Depart
                       from Employee E
                       where E.EmpId = MgrId ))
…
)
```

Assertions

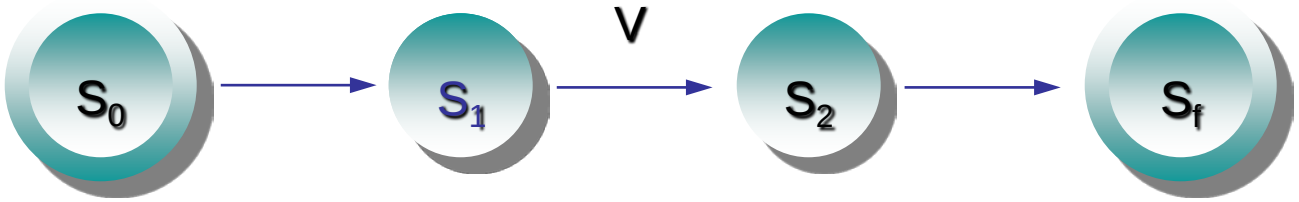
- Generic constraints outside the definitions of tables
- Basically, they consist in giving a (possibly meaningful) name to a check clause expressing a very generic constraint (typically cardinality constraints and ***inter-relational*** constraints)
- **Syntax**:
  create assertion *AssertionName* **check** (*Condition*)
- E.g., table Employee cannot be empty, and there should be no empty department:

```
create assertion ThereMustBeSomeoneEverywhere
  check ( 1 <= ( select count(*)
                 from Employee ) and
          not exists ( select * from
                       Department
                       where DepId not in
                             ( select Depart
                               from Employee ) )
```

Dynamically changing the constraint checking mode

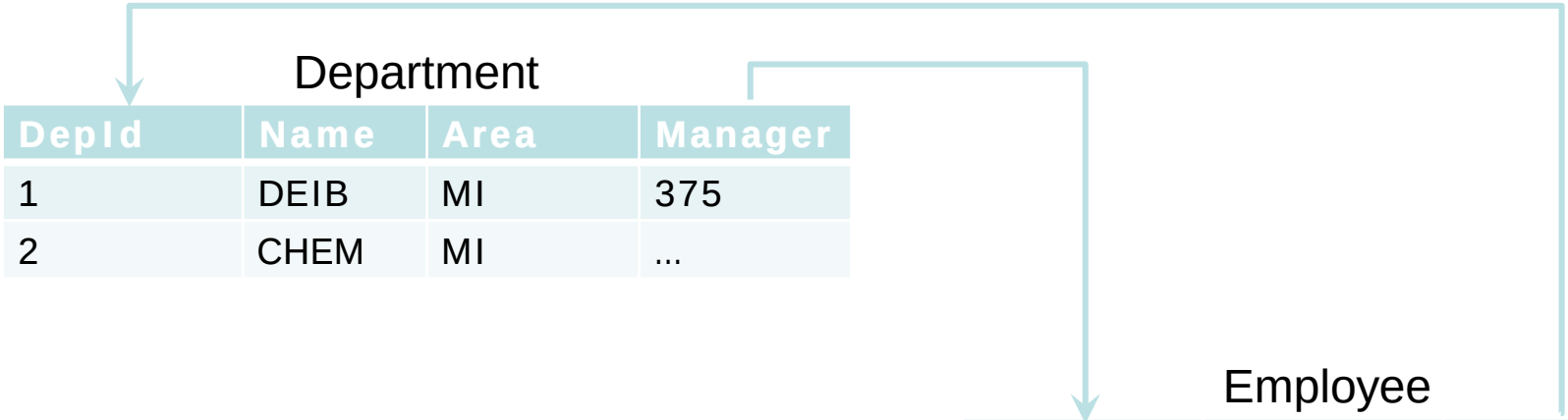
There are two checking modes for integrity constraints:

- **immediate** : any violation rejects the last modification
- **deferred** : any violation (verified at commit) rejects the entire transaction



- The default is *immediate*, but applications can change the verification mode:
  - `set constraints immediate`
  - `set constraints deferred`
- All constraints are checked in the end

Why is “set deferred” sometimes convenient?



Department

DepId	Name	Area	Manager
1	DEIB	MI	375
2	CHEM	MI	...

Employee

EmpId	Name	Depart
373	Paul	1
374	Richard	1
375	Mike	1
500	Johanna	2
501	Lucy	2

How to insert the first tuples

SET CONSTRAINTS DEFERRED

INSERT INTO Employee (375, Mike, 1)

INSERT INTO Department (1, DEIB, MI, 375)

SET CONSTRAINTS IMMEDIATE

INSERT INTO Employee (373, Paul, 1)

INSERT INTO Employee (374, Richard,1)

Stored Procedures

(«Methods» to encapsulate complex data behaviors)

Stored Procedures

- **SQL-2** defines ***stored procedure***, with a relatively limited expressive power
- Basically «code chunks» that carry out some (possibly delicate) data modifications
- Most commercial systems, however, offer powerful **extensions** (e.g., Oracle PL/SQL), with variables, instructions for flux control, iterators, etc.

→ Comparable to the expressive power of a full-fledged programming language

Life Cycle of a procedure

- Two distinct times:
  - declaration (...DDL)
  - invocation (...DML)
- DDL → stored procedure are part of the **schema**
  - They are “methods” relative to the dataset, specifying (and ***encapsulating***) their ***behavior***
- In a client-server architecture they are normally:
  - invoked by clients
  - stored and executed on the server

Example: Inventory Management

Inventory

ProdId	AvQty	ReordQty
1	150	100
3	130	80
4	170	50
5	500	150

PlannedReorders

ProdId	Date	ReordQty

- Users specify the Id and Quantity or the product to be picked from the inventory
- If the available quantity is insufficient, the withdrawal is aborted, and an exception is raised
- Otherwise, the required amount is subtracted from the available quantity
- If the modified available quantity is below the reorder quantity, then a reorder is planned

```
procedure Withdrawal
   (Prod integer, Quant integer) is
begin
 declare AQ, RQ integer;
 X exception;
 select AvQty, ReordQty into AQ, RQ
   from Inventory
   where ProdId = Prod;
 if AQ < Quant then raise(X);
   update Inventory
     set AvQty = AvQty - Quant
     where ProdId = Prod;
 if AQ - Quant < RQ then
    insert into PlannedReorders
     values(Prod,sysdate,RQ)
end;
```

Stored Procedures: PROs

- Modularity and Encapsulation
- Moreover:
  - Control (centralized, authoritative points of access)
  - Efficiency (data manioulations can be optimized «from within»)
  - Reuse (procedures are available to all the DB users)
- More responsibility for the DB administrators (as opposed to the Application programmers)
  - DBAdmins become responsible for a part of the application logic
  - Fragments of the application logic are moved from the apps to the DBMS (as long as they are related to inherent data properties)
- Some "knowledge" migrates from applications to the **schema** of the database
  - One step towards knowledge independence

Stored Procedures: CONs

- It is pointless to try and incapsulate too many possible data manipulations:
  - Procedures are efficient and effective if simple computations are required over large amounts of data
  - They are instead inefficient if they require are CPU-intensive computations and/or they target small amounts of data:
    - A DBMS is a DBMS in the end!

Active Databases

(Triggers)

Active Databases

The Trigger Concept

- Active databases: support for active rules (a.k.a. *triggers*)
- **ECA** Paradigm: **E**vent-**C**ondition-**A**ction
  - **whenever** an event **E** occurs
  - **if** a condition **C** is true
  - **then** an action **A** is executed
- An effective means to implement reactive computations
- Triggers in a DBMS:
  - "A set of actions that are automatically executed when an INSERT, UPDATE, or DELETE operation is performed on a specific table"
- Other examples of reactive behaviors in a DBMS:
  - Other generic business rules within database schemas
  - Integrity constraints (and reaction policies)
    - However, the «reaction» is merely rejecting the violation
- From «passive» to «really active» reactive behaviors

Active Databases

Event-Condition-Action

- **Event**
  - Normally a modification of the database status: **`insert`**, **`delete`**, **`update`**

  the trigger is ***activated***

- **Condition**
  - A predicate that identifies those situations in which the execution of the trigger's action is required

  the trigger is ***considered***

- **Action**
  - A generic insert/delete/update statement, a notification, or a stored procedure

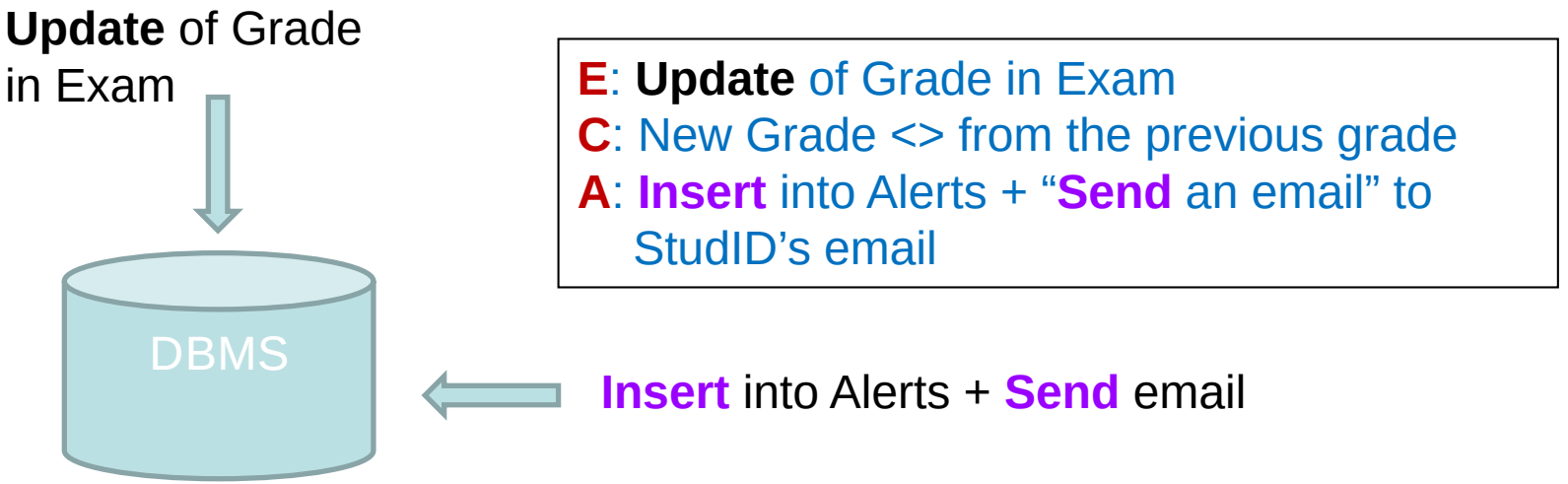
  the trigger is ***executed***

A simple example

```
Student(StudID, Name, Email, …)
Exam(StudID, CourseID, Date, Grade,…)
```

- All attempts are recorded in this schema
- When a grade is available (updated), the student is alerted, and the alert is recorded in the following table:

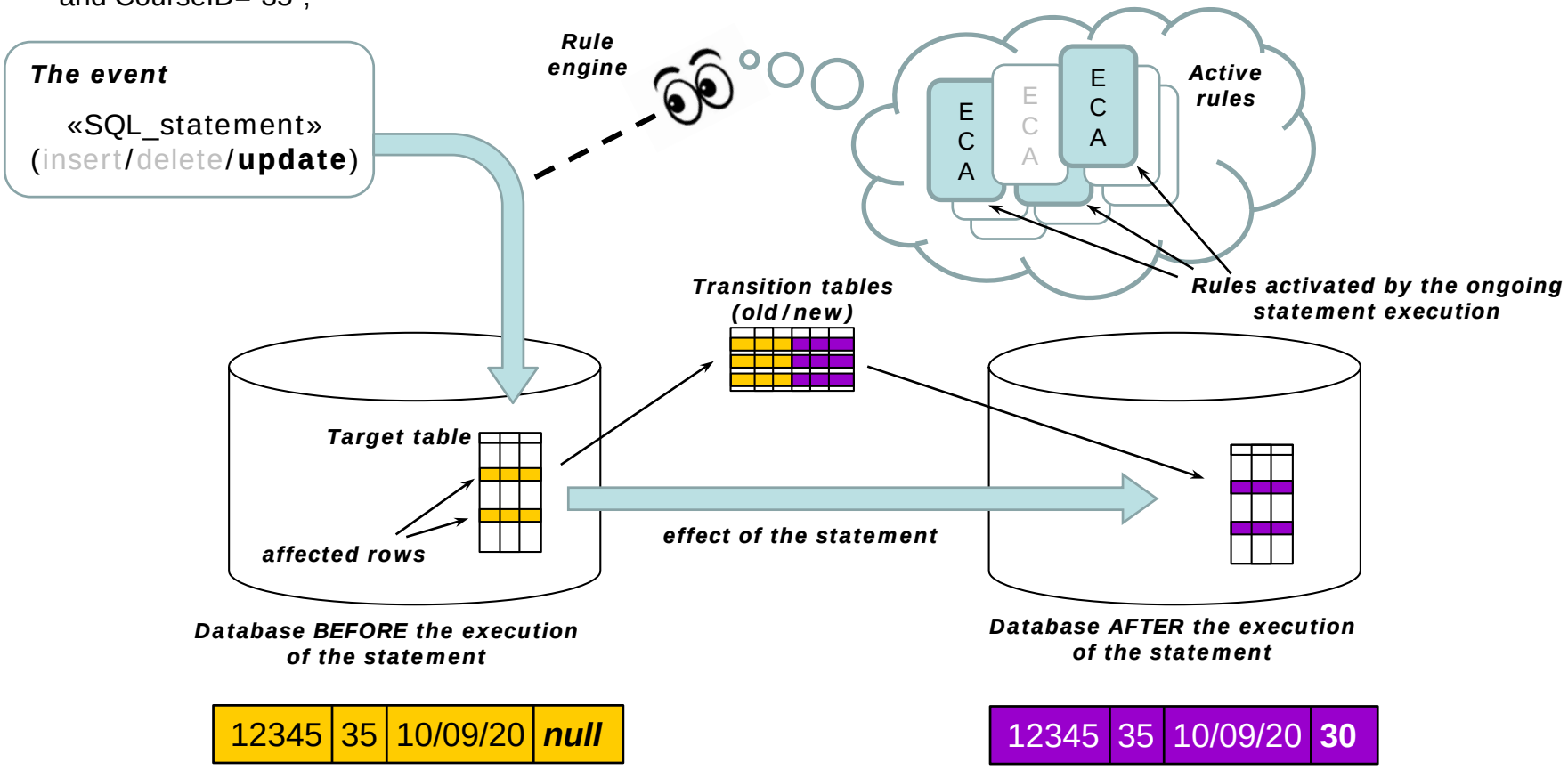
```
Alerts(TS, StudID)
```



Active Databases

The big picture

UPDATE Exam SET Grade="30"
WHERE **StudID** = **"12345"**
and CourseID="35";



Triggers in SQL:1999, Syntax

- SQL:1999 (aka SQL-3) was strongly influenced by DB2 (IBM)
  - the other systems, initially not fully compliant (as they exist since the mid eighties), tend to the standard
- Each trigger is characterized by (at least):

E
  - a name
  - an execution mode (**`before`** or **`after`**)
  - a monitored event (typically: **`insert`**, **`delete`**, or **`update`**)
  - the name of the target (monitored) table
  - a granularity (**`statement`**-level or **`row`**-level)

C
  - A condition (optional, if the trigger shall be always considered)
  - names and aliases for transition values and transition tables

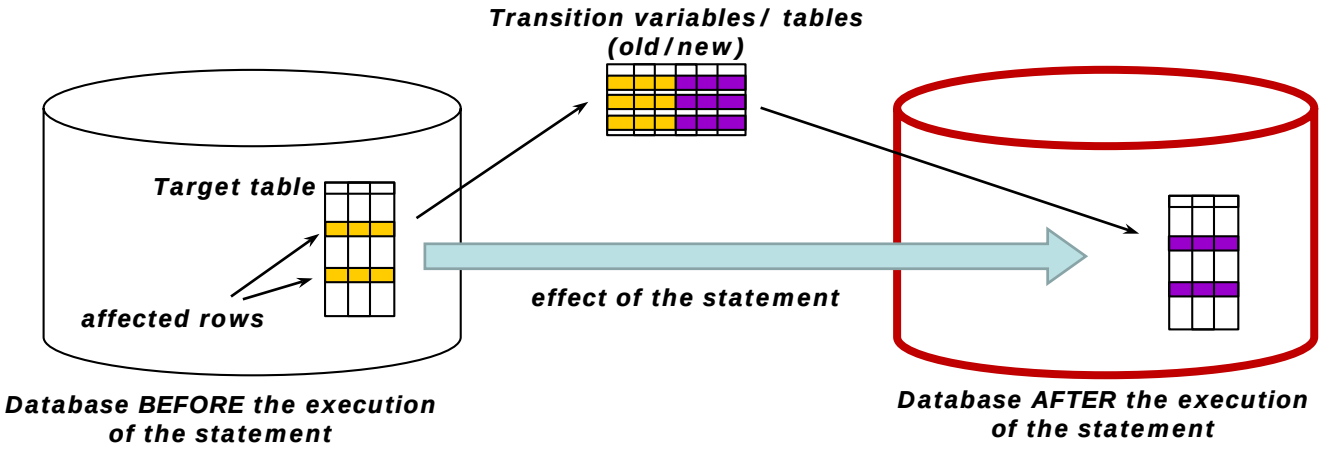
A
  - an action
  - creation timestamp

Triggers in SQL:1999, Syntax

```
create trigger <TriggerName>
{before | after}
E  {insert | delete | update [of <Column>]} on <Table>
   [referencing {[old table [as] <OldTableAlias> ]
                 [new table [as] <NewTableAlias> ] |
                 [old [row] [as] <OldTupleName> ]
                 [new [row] [as] <NewTupleName> ]} ]
   [for each {row | statement} ]
C  [when <Condition> ]
A  <SQLProceduralStatement>
```

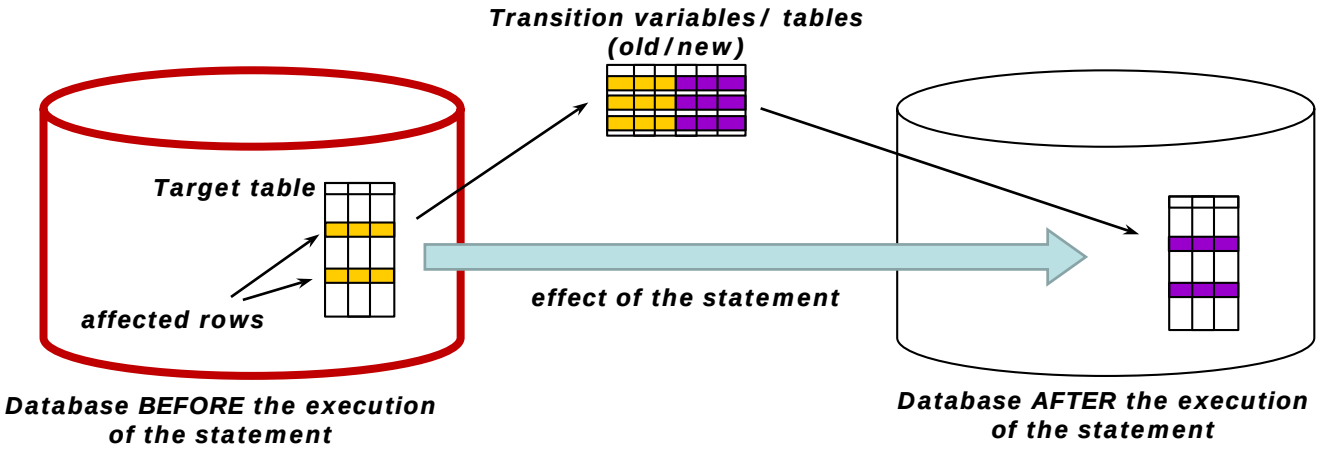
Execution modes: before or after

- **AFTER**
  - The trigger is *considered* (and possibly executed) **after the event**
    - i.e., *the condition is evaluated* on the DB as it is after the event
  - It is the most common mode, suitable for most applications



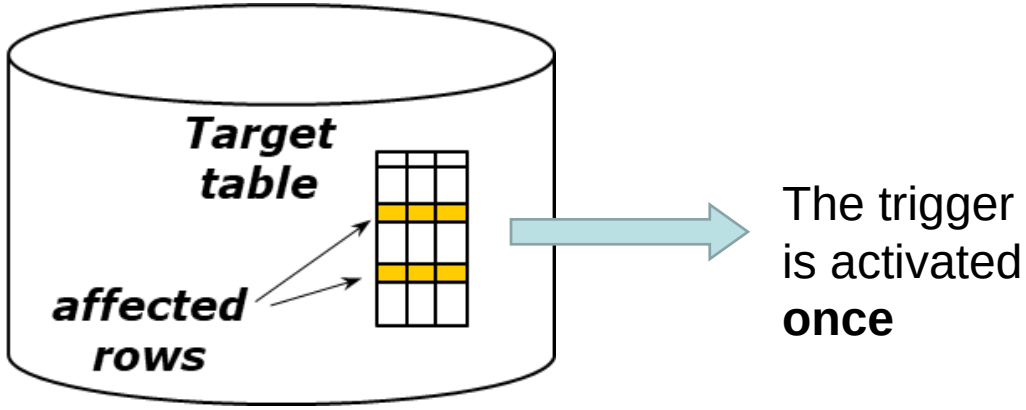
Execution modes: before or after

- **BEFORE**
  - The trigger is considered (and possibly executed) **before the event** is applied (i.e., before the database status change)
  - Typically, this mode is used to check and validate a modification before it takes place, and possibly condition the modification itself



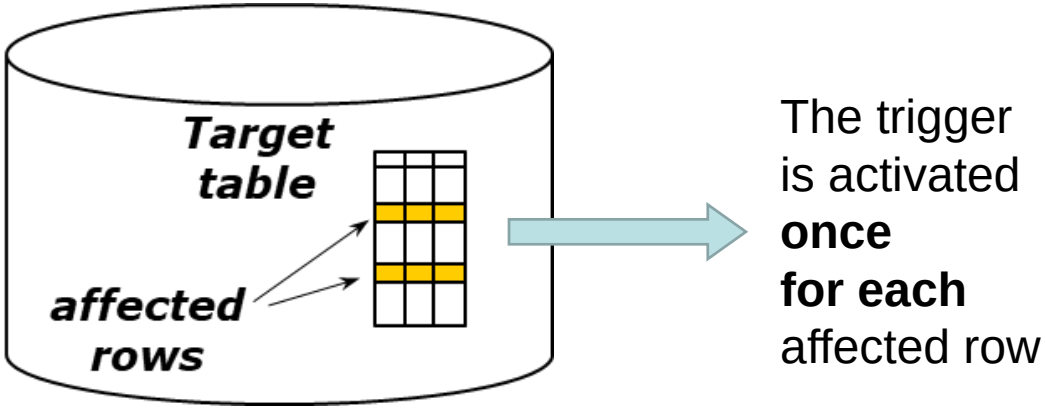
Safeness constraint: in many systems before triggers cannot update the database, but – as we will see - they can affect the transition variables in row-level granularity (*set new.attribute = <expr>*)

Granularity of events: row vs statement



- **Statement-level** granularity (default: **`for each statement`**)
  - The trigger is considered (and possibly executed) only once for each activating **statement**, independently of the number of affected tuples in the target table (even if no tuple is affected!)
  - Closer to the traditional approach of SQL statements, which are normally set-oriented

Granularity of events: row vs statement



- **Row-level** granularity (keyword: **`for each row`**)
  - The trigger is considered (and possibly executed) once for each **tuple** affected by the activating statement
  - Writing row-level triggers is simpler, but can be less efficient

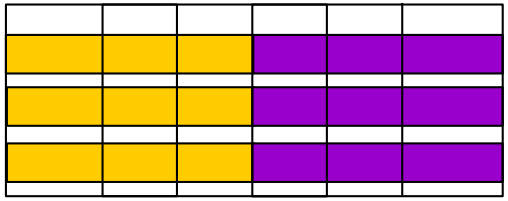
The variables / tables in the `referencing` clause

- Transition variables and transition tables enable tracking of the changes that activate triggers, and are crucial to efficiency
- Its syntax descends from the chosen granularity
  - If it is row-level, two ***transition variables*** (**`old`** and **`new`**) represent the value respectively prior to and following the modification of the row (i.e., tuple) under consideration
  - If it is statement-level, two ***transition tables*** (**`old table`** and **`new table`**) contain respectively the old and the new value of all the affected rows (tuples)
- Variables **`old`** and **`old table`** are undefined in triggers whose event is **`insert`**
- Variables **`new`** and **`new table`** are undefined in triggers whose event is **`delete`**

Transition variable (old / new)



Transition tables (old table / new table)



Active Databases

A row-level trigger and a statement-level trigger

Account(AccNumber, Customer, Balance, …)

IncomingPayments(AccNumber, Amount, Date, Time)

```
create trigger AccountMonitor
after update of Balance on Account
for each row
when new.Balance > old.Balance
insert into IncomingPayments
  values( new.AccNumber, new.Balance-old.Balance,
          sysdate(), systime() )
```

Invoice(InvNum, IssueDate, DueDate, Description, Amount, Status)

DeletedInvoice(InvNum, IssueDate, DueDate, Description, Amount, Status)

```
create trigger FilingOfDeletedInvoices
after delete on Invoice
insert into DeletedInvoice ( select *
                             from old table )
```

Active Databases

A simple example: mirroring

- Table T1:

ID	VALUE
1	10
2	15

- Table T2:

ID	VALUE
1	10
2	15

- ID is the primary key
- Table T2 is a replication of table T1: whenever a modification occurs on T1, this is replicated on T2
- Replication is realized by means of triggers

Insertion of a new tuple into T1

- Table T1:

ID	VALUE
1	10
2	15
3	**20**

- Table T2:

ID	VALUE
1	10
2	15
3	**20**

TRANSITION VARIABLE

old

new

ID	VALUE
3	**20**

```
CREATE TRIGGER REPLIC_INS
AFTER INSERT ON T1
FOR EACH ROW
INSERT INTO T2 VALUES (new.ID, new.VALUE);
```

Active Databases

Deletion of a tuple from T1

- Table T1:

ID	VALUE
1	10
~~2~~	~~15~~
3	20

- Table T2:

ID	VALUE
1	10
~~2~~	~~15~~
3	20

TRANSITION VARIABLE

old

ID	VALUE
2	15

new

```
CREATE TRIGGER REPLIC_DEL
AFTER DELETE ON T1
FOR EACH ROW
DELETE FROM T2 WHERE T2.ID = old.ID;
```

Update of a tuple in T1

- Table T1:

ID	VALUE
1	~~10~~ 5
2	15
3	20

- Table T2:

ID	VALUE
1	~~10~~ 5
2	15
3	20

TRANSITION VARIABLES

old		new	
ID	VALUE	ID	VALUE
1	10	1	5

```
CREATE TRIGGER REPLIC_UPD
AFTER UPDATE OF VALUE ON T1
FOR EACH ROW
UPDATE T2 SET T2.VALUE = new.VALUE
   WHERE T2.ID = new.ID;
```

Triggers with a condition: conditional mirroring

- Variant: Table T2 is a replication of table T1, but only the tuples whose value is >= 10 are replicated.
- Insertion operation:

```
CREATE TRIGGER REPLIC_INS
AFTER INSERT ON T1
FOR EACH ROW
WHEN (new.VALUE >= 10)
INSERT INTO T2 VALUES (new.ID, new.VALUE);
```

The other triggers shall be adapted too:

- the one for deletion has an analogous condition
- the one for updates shall consider 3 options:

8 → 12 : *insert* 11 → 13 : update 14 → 7 : delete

FOR EACH ROW vs. FOR EACH STATEMENT / 1

- A set of tuples is deleted from T1:

```
DELETE FROM T1
WHERE VALUE>10;
```

ID	VALUE
1	10
~~2~~	~~15~~
~~3~~	~~20~~

old (first activation)

ID	VALUE
2	15

old (second activation)

ID	VALUE
3	20

```
CREATE TRIGGER REPLIC_DEL
AFTER DELETE ON T1
FOR EACH ROW    (one row at a time is considered!)
DELETE FROM T2 WHERE T2.ID = old.ID;
```

FOR EACH ROW vs. FOR EACH STATEMENT /2

- A set of tuples is deleted from T1:

```
DELETE FROM T1
WHERE VALUE>10;
```

ID	VALUE
1	10
~~2~~	~~15~~
~~3~~	~~20~~

old table

ID	VALUE
2	15
3	20

```
CREATE TRIGGER REPLIC_DEL
REFERENCING OLD TABLE AS OLD_HIGHVALUES
AFTER DELETE ON T1
FOR EACH STATEMENT (all the tuples are considered at once)
DELETE FROM T2 WHERE T2.ID IN
        (SELECT ID FROM OLD_HIGHVALUES);
```

Example of BEFORE trigger

- Extension: *no tuples with negative values can be <u>inserted</u> (action: reject the insertion)*

```
CREATE TRIGGER NO_NEGATIVE_VALUES
BEFORE INSERT ON T1
FOR EACH ROW
WHEN (new.VALUE < 0)
raise_application_error (....);
```

old

new

ID	VALUE
5	-8

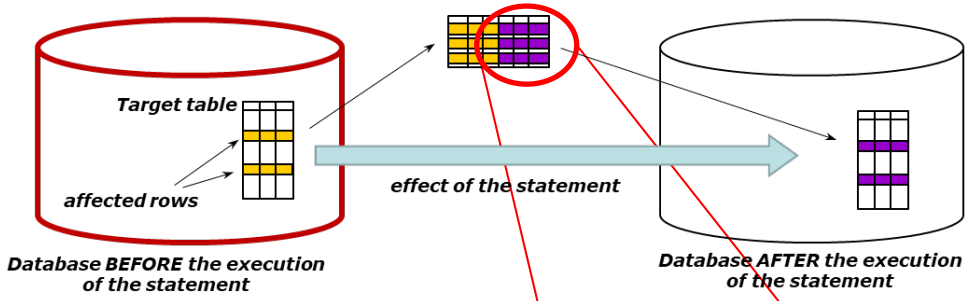
(for the update operation a similar trigger is defined)

Example of BEFORE trigger with a «Conditioner»

- Extension: *prevent values to be <u>updated</u> to negative values (in such a case, the action is to **set** the value **to 0**)*

Two versions:

- 1. BEFORE mode

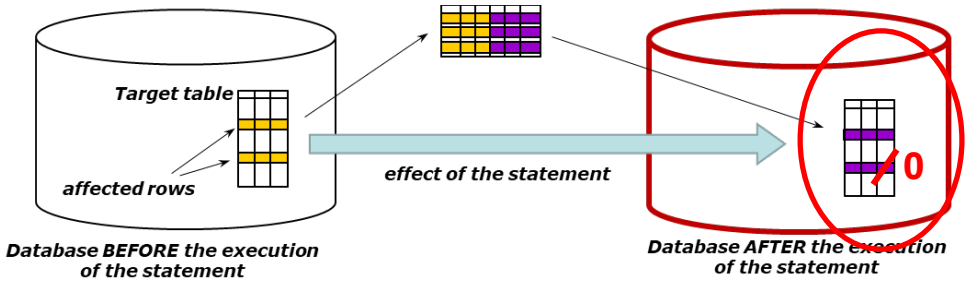



```
CREATE TRIGGER NO_NEGATIVE_VALUES
    BEFORE UPDATE of VALUE ON T1
    FOR EACH ROW
    WHEN (new.VALUE < 0)
    SET new.VALUE=0;
```

old		new	
ID	VALUE	ID	VALUE
5	6	5	~~-3~~ 0

The same result with an AFTER trigger (a «re-installer»)

- 2. AFTER mode




```
CREATE TRIGGER NO_NEGATIVE_VALUES
AFTER UPDATE of VALUE ON T1
FOR EACH ROW
WHEN (new.VALUE < 0)

UPDATE T1
    SET VALUE=0
    WHERE ID=new.ID;
```

old		new	
ID	VALUE	ID	VALUE
5	6	5	-8

Another example of a trigger in before mode

- Prevents salaries to be increased by more than 20% within a single update operation
- Presented in two versions: 1. before mode 2. after mode
- 1. "**conditioner**" (acts before the update and integrity checking)

```
create trigger Max20percent_before
before update of Salary on Employee
for each row
when new.Salary > old.Salary * 1.2
set new.Salary = old.Salary * 1.2
```



*the **transition variable** is modified*

And again, the same result with a trigger in after mode

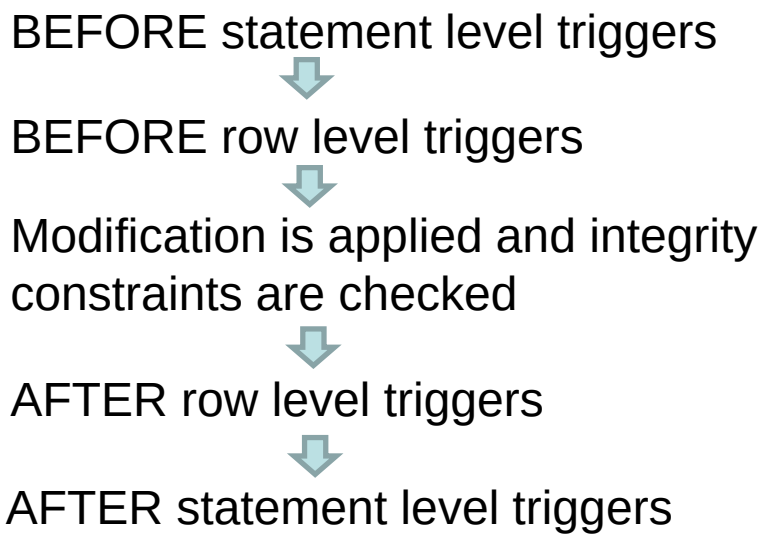
- 2. "**re-installer**" (acts after the update)

```
create trigger Max20percent_after
after update of Salary on Employee
for each row
when new.Salary > old.Salary * 1.2
update Employee
  set Salary = old.Salary * 1.2
    where RegNum = new.RegNum
```



Execution of Multiple Triggers: Conflicts

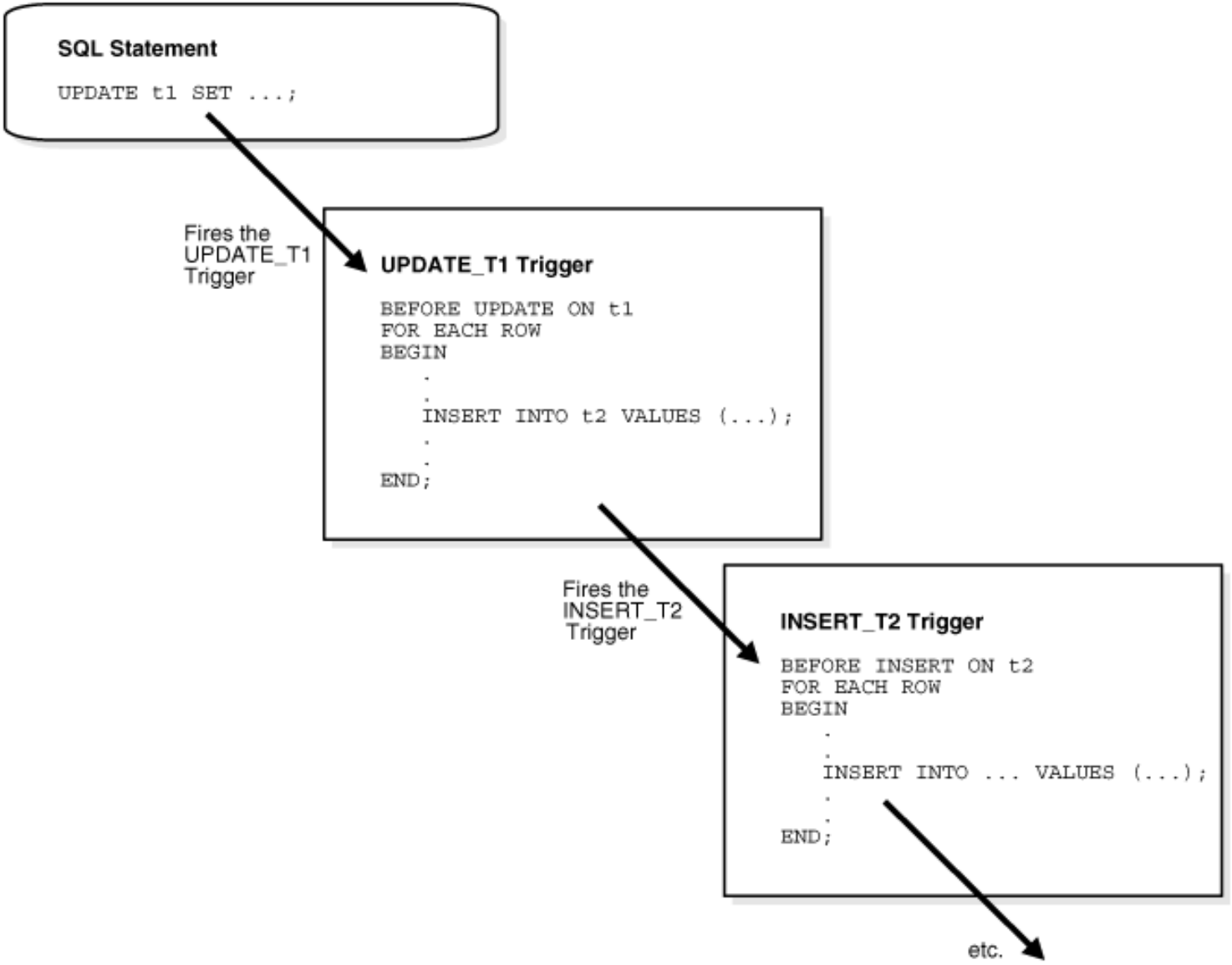
- If several triggers are associated with the same event, SQL:1999 prescribes the following execution policy:



- If there are several triggers in the same category, the order of execution depends on the system implementation (e.g., based on the definition time (older triggers have higher priority))

Active Databases

Also, the action part of a rule can trigger other rules!



Design - Trigger Properties

- It is important to ensure that **interferences** among triggers and **chain activations** do not produce undesired system behaviors
- Three classical properties
  - **Termination**: for any initial state and any sequence of modifications, a final state is always produced (infinite activation cycles are not possible)
  - Confluence: triggers terminate and produce a unique final state, independent of the order in which triggers are executed
    - Meaningful only if there is nondeterminism in the activation
- Termination is by far the most important property

Example with Two Triggers

Employee (RegNum, Name, Salary, Contribution, DeptN, ...)

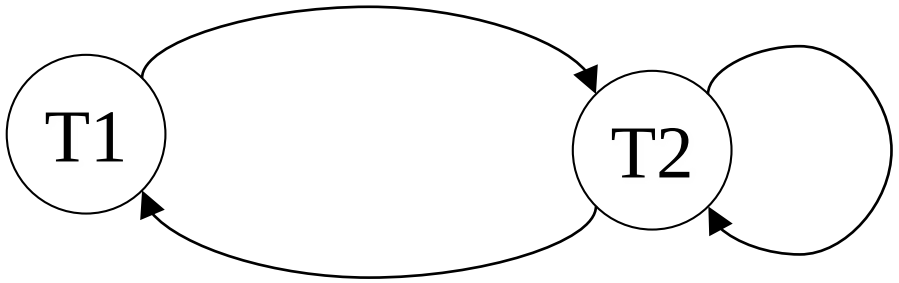
```
T1:    create trigger AdjustContributions
       after update of Salary on Employee
       referencing new table as NewEmp
       update Employee
        set Contribution = Salary * 0.8
         where RegNum in ( select RegNum from NewEmp )

T2:    create trigger CheckOverallBudgetThreshold
       after update on Employee
       when 50000 < ( select sum(Salary+Contribution)
                      from Employee )
       update Employee
        set Salary = 0.9 * Salary;
```

Termination Analysis

- There exist several tools, most of which based on graphs
- The simplest abstraction is the **triggering graph**
  - A node i for each trigger t_i
  - An arc from a node i to a node j if the execution of trigger t_i's action **may** activate trigger t_j
  - The graph is built with a simple syntactic analysis
- If the graph is acyclic, the system is guaranteed to terminate
  - There cannot be infinite trigger sequences
- If the graph has some cycles, it *may* be non-terminating
  - but it might as well be terminating (cfr.: recursion in PLs)

Triggering Graph for the previous triggers



- There are two cycles, but the system is terminating
- What if you invert the comparison in T2's condition?

```
when 50000 > (…)
```

Another Example: Salary Management

Employee

RegNum	Name	Salary	DeptN	ProjN
50	Smith	5.900	1	20
51	Black	5.600	1	10
52	Jones	5.000	1	20

Department

DeptNum	MGRRegNum
1	50

Project

ProjNum	Crucial
10	no
20	no

Example Trigger T1: Bonus

Event: **update** of the **Crucial** attribute in table **Project**
Condition: New value: C**rucial = 'yes'**
Action: Increase by 10% the salary of the employees involved in the project that becomes crucial

```
create trigger Bonus_T1
after update of Crucial on Project
for each row
when new.Crucial = 'yes' and old.Crucial = 'no'
update Employee
   set Salary = Salary * 1.10
   where ProjNum = new.ProjNum;
```

Example Trigger T2: CheckIncrement

Event: **update** of **Salary** in **Employee**
Condition: The new salary is greater than the manager's salary
Action: Decrease salary and make it the same as the manager's

```
create trigger CheckIncrement_T2
after update of Salary on Employee
for each row
declare X number;
begin
   select Salary into X
   from Employee join Department D on RegNum = D.MGRRegNum
   where D.DeptNum = new.DeptN;

   if( new.Salary > X ) update Employee set Salary = X
                        where RegNum = new.RegNum;       endif;
end;
```

Example Trigger T3: CheckDecrement

Event: **update** of **Salary** in **Employee**
Condition: Decrement greater than 3%
Action: Decrement salary only by 3%

```
create trigger CheckDecrement_T3
after update of Salary on Employee
for each row
when new.Salary < old.Salary * 0.97
update Employee
   set Salary = old.Salary * 0.97
   where RegNum = new.RegNum;
```

Activation of T1

```
update Project
  set Crucial = 'yes'
    where ProjNum = 10
```

Event: update of **Crucial** in **Project**

Condition: true

Action: increase Black's salary by 10%

Project

ProjNum	Crucial
10	**yes**
20	no

Employee

RegNum	Name	Salary	DeptN	ProjN
50	Smith	5.900	1	20
51	Black	**6.160**	1	10
52	Jones	5.000	1	20

Activation of T2

Event: update of **Salary** in **Employee**

Condition: true (Black's salary is greater than Smith's)

Action: Black's salary is set to Smith's

RegNum	Name	Salary	DeptN	ProjN
50	Smith	5.900	1	20
51	Black	**5.900**	1	10
52	Jones	5.000	1	20

- T2 is activated again – the condition in false (not increased)
- T3 is activated

Activation of T3

Event: update of **Salary** in **Employee**

Condition: true (Black's salary was decreased by more than 3%)

Action: Black's salary is decreased by only 3%

RegNum	Name	Salary	DeptN	ProjN
50	Smith	5.900	1	20
51	Black	**5.975,20**	1	10
52	Jones	5.000	1	20

- T3 is activated again – the condition is false (not decreased)
- T2 is activated again – the condition is true (increased)

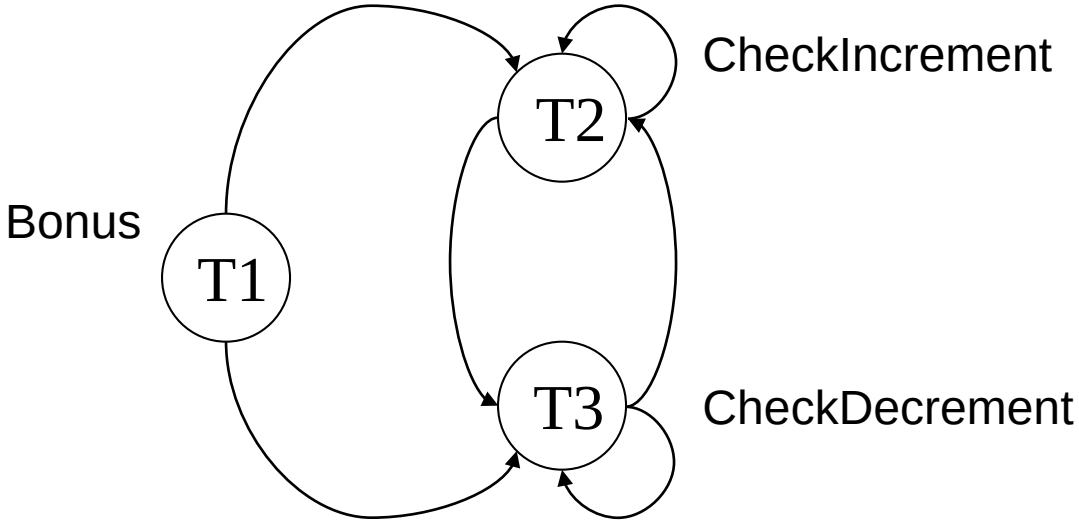
Activation of T2

RegNum	Name	Salary	DeptN	ProjN
50	Smith	5.900	1	20
51	Black	**5.900**	1	10
52	Jones	5.000	1	20

Activation of T3

- The trigger condition is false
  - This time, the salary was decreased by less than 3% - T3 is overall ineffective!
- Trigger activation has reached termination

Termination Graph for Salary Management



- The graph is cyclic, but the repeated execution of the triggers reaches termination anyway

A simple example (Book-Author)

Consider the following relational schema:

BOOK(Isbn, Title, SoldCopies)
WRITING(Isbn, Name)
AUTHOR(Name, SoldCopies)

Define a set of triggers for keeping SoldCopies in AUTHOR updated with respect to:

- updates on SoldCopies in BOOK
- insertion of new tuples in the WRITING relation

A simple example (Book-Author)

```
create trigger UpdateSalesAfterNewSaleOfExistingBook
after update of SoldCopies on BOOK
for each row
update AUTHOR
  set SoldCopies = SoldCopies +
                          + new.SoldCopies – old.SoldCopies
  where Name in ( select Name
                  from WRITING
                  where Isbn = new.Isbn)
```

WARNING: In the assignment, Sold copies and **new**.SoldCopies are not the same thing… they correspond to *Author*.SoldCopies and *Book*.SoldCopies respectively, that are two distinct concepts

A simple example (Book-Author)

```
create trigger UpdateSalesAfterDiscoveringNewAuthorship
after insert on WRITING
for each row
update AUTHOR
  set SoldCopies = SoldCopies +
                                ( select SoldCopies
                                  from BOOK
                                  where Isbn = new.Isbn )
where Name = new.Name
```

*It may seem that we should also react to the insertion of new books. However, for the authorship to be granted, **a tuple in Writing is required too**, and we already have a trigger to update the counter in that case, therefore such a trigger should not be added.*

Trigger Applications

- Triggers add powerful data management capabilities in a transparent and reusable manner
  - Databases can be enriched with “business and management rules” that would otherwise be distributed over all applications
- However, understanding the interactions between triggers is rather complex: triggers are an underexploited feature
- DBMS vendors use triggers to implement internal services
  - Examples:
    - Data replication (or computation of derived data)
    - Management of constraints (e.g., referential integrity)
    - View maintenance (for materialized views)

Applications of Active Databases

- **Internal** rules (system-generated and not visible to users)
  - Integrity constraint management
  - Computation of derived and replicated data
  - Versioning management, privacy, security,
  - Action logging, event recording
- **External** rules (generated by database administrators, express application-specific knowledge)
  - Personalization, adaptation
  - Context-awareness
  - Business rules

Referential Integrity Management

- *Repair strategies* for violations of referential integrity constraints

```
CREATE TABLE Employee (
… …
FOREIGNKEY(DeptN) REFERENCES Department(DeptNum)
  ON DELETE SET NULL  ON UPDATE CASCADE,
… … );
```

Operations that can violate this constraint:

- **INSERT** into Employee
- **UPDATE** of Employee.DeptN
- **UPDATE** of Department.DeptNum
- **DELETE** from Department

Employee

RegNum	…	Salary	DeptN
1		100	1
2		80	2
3		74	1
4		90	1
5		88	2

Department

DeptNum	MGRRegNum
1	1
2	5

Actions in the Employee Table

Event: insert into **Employee**
Condition: the new **DeptN** value is not in the **Dept** table
Action: **no action** policy: insertion is inhibited, reporting error

```
create trigger CheckEmpDept
before insert on Employee
for each row
when not exists ( select * from Department
                  where DeptNum = new.DeptN )
raise_application_error(-20000, 'Invalid Department');
```

- The trigger for the *update* of **DeptN** in **Employee** is analogous (identical but for **before update** instead of **before insert**)

  [*à-la-Oracle: rollback is automatically done by* `raise_application_error`]

Deletion in the Department Table

Event: delete from **Department**
Condition: the deleted **DeptNum** is used in the **Employee** table
Action: **set null** policy (the employee's **DeptN** is set to null)

```
create trigger CheckDeptDeletion
after delete on Department
for each row
when exists ( select * from Employee
              where DeptN = old.DeptNum )
update Employee set DeptN = null
  where DeptN = old.DeptNum;
```

(note: condition could be omitted)

Updates in the Department Table

Event: update of **DeptNum** in **Department**
Condition: the old **DeptNum** value is used in the **Employee** table
Action: **cascade** policy (**DeptN** in Employee is also modified)

```
create trigger CheckDeptUpdate
after update of DeptNum on Department
for each row
when exists ( select * from Employee
              where DeptN = old.DeptNum )
update Employee set DeptN = new.DeptNum
   where DeptN = old.DeptNum;
```

(note: condition could be omitted)

Triggers for Materialized View Maintenance

- Materializing a view is convenient when its usage is more frequent than the updates (on the base tables) that change its result
- A naïve approach to keep the view up-to-date consists in fully re-calculating the view whenever one of the base tables is modified
- A simple example: overall personnel cost of each department:

```
CREATE MATERIALIZED VIEW PersonnelDepCost(DpN,SalarySum) AS
select DeptN, sum(Salary)
from Employee
group by DeptN;
```

- The view shows the personnel cost of every non-empty department

Employee

RegNum	Salary	DeptN
1	100	1
2	80	2
3	74	1
4	90	1
5	88	2

PersonnelDepCost

DpN	SalarySum
1	264
2	168

Triggers for Materialized View Maintenance

- Triggers for the **naïve approach** (**statement-level**)

```
create trigger NaiveRecompute_upd(|_del|_ins)
after update(|delete|insert) on Employee
begin
  delete from PersonnelDepCost;
  insert into PersonnelDepCost
    select DeptN, sum(Salary)
    from Employee
    group by DeptN;
end;
```

Three almost identical triggers: whatever the modification is, the whole materialization is dropped and reinstalled

Triggers for Materialized View Maintenance

- However, not all modifications onto Employee affect the view, and most modifications only affect a small part of the materialized data:
  - Update of **RegNum**, **Name**, or **ProjN**: **no effect** on **PersonnelDepCost**
  - Insertion of a new Employee: only affects one tuple in **PersonnelDepCost**
  - Deletion of an Employee: only affects one tuple in **PersonnelDepCost**
  - Update of one **Salary**: only affects one tuple in **PersonnelDepCost**
  - Update of one **DeptN**: only affects **two** tuples in **PersonnelDepCost**
- In all these cases, and whenever the computational effort required to deal with the delta in the view is significantly smaller than that of fully recomputing the view, an ***incremental* approach** is preferrable
  - A higher number of (more specific) triggers may be required

RegNum	Salary	DeptN
1	100	1
2	80	2
3	74	1
4	90	1
5	88	2

DpN	SalarySum
1	264
2	168

Materialized View Maintenance: Incremental approach

- Incremental approach: Insertion and deletion of employees

```
create trigger Incremental_InsEmp
after insert on Employee
for each row
update PersonnelDepCost
 set SalarySum = SalarySum + new.Salary
  where DpN = new.DeptN;

create trigger Incremental_DelEmp
after delete on Employee
for each row
update PersonnelDepCost
 set SalarySum = SalarySum – old.Salary
  where DpN = old.DeptN;
```

***row-level** triggers !*

If more than one employee is inserted(deleted) by the same SQL command, each affected tuple affects only one department in the materialized view

RegNum	Salary	DeptN
1	100	1
2	80	2
3	74	1
4	90	1
5	88	2
6	75	2

DpN	SalarySum
1	264
2	243

Materialized View Maintenance: Incremental approach

- Incremental approach: update of **Salary**

```
create trigger Incremental_SalaryUpdate
after update of Salary on Employee
for each row
update PersonnelDepCost
 set SalarySum = SalarySum + new.Salary – old.Salary
  where DpN = new.DeptN;
```

Applies the "salary delta"...

...only to the department to which the updated employee is affiliated (using old.DeptN would be equally correct, as this is not changed)

RegNum	Salary	DeptN
1	100	1
2	80	2
3	74	1
4	90	1
5	88	2
6	~~75~~ 80	2

DpN	SalarySum
1	264
2	~~243~~ 248

Materialized View Maintenance: Incremental approach

- Incremental approach: update of **DeptN** (moving the employee)

```
create trigger Incremental_DeptChange
after update of DeptN on Employee
for each row
begin
  update PersonnelDepCost
    set SalarySum = SalarySum + new.Salary
      where DpN = new.DeptN;
  update PersonnelDepCost
    set SalarySum = SalarySum – old.Salary
      where DpN = old.DeptN;
end;
```

The sum is incremented (resp. decremented) in the new (resp. old) department

RegNum	Salary	DeptN
1	100	1
2	80	2
3	74	1
4	90	1
5	88	2
6	80	~~2~~ 1

DpN	SalarySum
1	~~264~~ 344
2	~~248~~ 168

View Maintenance: the role of Integrity

- The previous incremental triggers are correct only if
  - The modifications do not violate the integrity constraints
  - The previous materialization is already correct
  - The other tables (other than Employee) are not modified

  otherwise, the resulting materialization may not be aligned with the base table
- Instead, the “naïve” approach is less error prone w.r.t. this aspect
- Example: we didn’t consider the **creation of a new department**!
  - Employees moved to the new department would subtract their salary from their old affiliation but wouldn’t add it to the new one, that cannot be present in the materialization
  - Some trigger(s) should be responsible for creating a new tuple in the materialization if an employee is assigned to a *new* department

Incremental View Maintenance: the devil and the details

- Some trigger(s) should be responsible for creating a new tuple in the materialization if an employee is assigned to a *new* department
  - This assignment may occur by means of an *insertion* of a new employee or by means of an *update* to the current department of an existing employee
  - Thus, both triggers **Incremental_InsEmp** and **Incremental_DeptChange** shall be extended in order to deal with this issue:

```
create trigger Incremental_InsEmp_revised
after insert on Employee
for each row
if( new.DeptN in ( select DpN
                   from PersonnelDepCost ) )
then update PersonnelDepCost
      set SalarySum = SalarySum + new.Salary
       where DpN = new.DeptN;
else insert into PersonnelDepCost
      values( new.DeptN, new.Salary );
```

RegNum	Salary	DeptN
1	100	1
2	80	2
3	74	1
4	90	1
5	88	2
6	80	2
7	85	3

DpN	SalarySum
1	344
2	168
3	85

Incremental View Maintenance: the devil and the details

```
create trigger Incremental_DeptChange_revised_1
after update of DeptN on Employee
for each row
begin
  update PersonnelDepCost
    set SalarySum = SalarySum – old.Salary
      where DpN = old.DeptN;
  if( new.DeptN in ( select DpN
                     from PersonnelDepCost ) )
  then  update PersonnelDepCost
          set SalarySum = SalarySum + new.Salary
            where DpN = new.DeptN;
  else  insert into PersonnelDepCost
          values( new.DeptN, new.Salary );
end;
```

- Here the old department has its cost decreased in both cases
  - But… what if the moved employee was the last one of the department?
  - In that case, the materialization would get a tuple with ***zero cost***, that cannot be generated by the view (unless some employee has salary equa to zero)
  - The trigger, then, must be further extended!

Incremental View Maintenance: the devil and the details

```
create trigger Incremental_DeptChange_revised_2
after update of DeptN on Employee
for each row
begin
  if( exists ( select * from Employee
               where DpN = old.DeptN ) )
  then  update PersonnelDepCost
          set SalarySum = SalarySum – old.Salary
            where DpN = old.DeptN;
  else  delete from PersonnelDepCost
          where DpN = old.DeptN;
  if( new.DeptN in ( select DpN
                     from PersonnelDepCost ) )
  then  update PersonnelDepCost
          set SalarySum = SalarySum + new.Salary
            where DpN = new.DeptN;
  else  insert into PersonnelDepCost
          values( new.DeptN, new.Salary );
end;
```

RegNum	Salary	DeptN
1	100	1
2	80	2
3	74	1
4	90	1
5	88	2
6	80	2
7	85	3

1

DpN	SalarySum
1	344
2	168
3	85

429

- Analogously, in case of *deletion* of the last employee of a department there should be no tuple with zero cost left in the view, and trigger **Incremental_DelEmp** should be revised too

Incremental View Maintenance: the devil and the details

```
create trigger Incremental_DelEmp_revised
after delete on Employee
for each row
  if( exists ( select * from Employee
               where DpN = old.DeptN ) )
  then  update PersonnelDepCost
          set SalarySum = SalarySum – old.Salary
            where DpN = old.DeptN;
  else  delete from PersonnelDepCost
          where DpN = old.DeptN;
```

RegNum	Salary	DeptN
1	100	1
2	80	2
3	74	1
4	90	1
5	88	2
6	80	2
~~7~~	~~85~~	~~3~~

DpN	SalarySum
1	344
2	168
~~3~~	~~85~~

- Now the materialization is correctly maintained incrementally w.r.t. the definition of the view
- *Please note that most of the complications arise from the fact that the view does not include the “zero” cost of any department with no employees*
  - *A more «inclusive» view definition would make the maintenance easier*

Incremental View Maintenance: smaller devil, fewer details

- *A more «inclusive» view definition would make the incremental maintenance easier*

```
CREATE MATERIALIZED VIEW PersonnelDepCost(DpN,SalarySum) AS
select DeptN, ( select coalesce( sum(Salary), 0 )
                from Employee
                where DeptNum = D.DeptNum )
from Department D;
```

function returns the first non-null value in a list

- Now the materialization also include “zero” tuples for empty departments
- The initial set of four triggers can be (more easily) extended with two dedicated triggers that insert and delete the tuples of the view materializations whenever departments are added or removed

View Maintenance: the role of Integrity

- Two dedicated triggers shall create a new tuple in the materialization for the new department, and delete such tuple when an (empty!) department is deleted:

```
create trigger Incremental_NewDept
after insert on Department
insert into PersonnelDepCost
 select DeptNum, 0
 from new table;
```

These triggers are statement-level so as to compactly react also to «bulk» deletion/insertions.

It is correct to insert the new tuple with 0 as initial value for SalarySum.

```
create trigger Incremental_NewDept
after delete on Department
delete from PersonnelDepCost
 where DpN in ( select DeptNum
                from old table );
```

It is not necessary to check that SalarySum is 0 before deletion, as this should be already granted by the overall correctness of the process.

- Of course, this is because we are assuming that there exists a referential integrity from Employee(DeptN) to Department(DeptN) (and therefore only empty departments can be inserted or deleted).